The Journal of Open Source Software

pyrdm: A Declarative Object-RDF Mapper for Python

Felipe dos Santos Goiabeira[1] **and Sergio Souza Costa**[1]

1 Federal University of Maranhão (UFMA), Brazil

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

pyrdm is a Python library that provides declarative Object-RDF Mapping (ORM), enabling developers to map Python classes to RDF graphs (W3C RDF Working Group, 2014) using a decorator-based API without writing SPARQL (W3C SPARQL Working Group, 2013) queries or manipulating triples directly. The library is inspired by established ORM frameworks such as JPA (Eclipse Foundation, 2020) and SQLAlchemy (Bayer, 2012), adapted to the requirements of the Semantic Web (Heath & Bizer, 2011).

Built on top of `rdflib` (Team, 2023) and `pyshacl` (Sommer, 2024), `pyrdm` bridges the gap between object-oriented programming and RDF-based knowledge representation. It handles serialization and deserialization of Python objects to and from RDF graphs, generates SHACL shapes (W3C RDF Data Shapes Working Group, 2017) automatically from class metadata, and provides a dynamic repository API for querying entities by field values without writing SPARQL.

Statement of Need

The Semantic Web ecosystem provides powerful standards — RDF, RDFS, OWL, SPARQL, and SHACL — for representing and querying structured knowledge. However, integrating these technologies into Python applications typically requires developers to work directly with low-level graph APIs such as `rdflib` (Team, 2023), managing URIs, triples, and SPARQL queries explicitly. This creates a significant productivity barrier, especially for researchers and developers who are domain experts but not Semantic Web specialists.

The idea of bridging object-oriented models and graph data is not new. Fowler's patterns of enterprise application architecture (Fowler, 2002) established the ORM as a foundational design pattern for relational databases, later formalized in Java through the JPA specification (Eclipse Foundation, 2020) and in Python through SQLAlchemy (Bayer, 2012). In the RDF world, Empire (Bischof et al., 2011) brought a JPA-style annotation API to Java, and RDFAlchemy (Dodds, 2012) provided an early attempt at ORM-style access to RDF stores in Python. However, RDFAlchemy has been unmaintained since 2012 and does not support modern Python packaging, type annotations, or SHACL validation.

`pyrdm` fills this gap for the Python ecosystem by providing:

- A decorator API (`@rdf_entity`, `@rdf_property`, `@rdf_one_to_one`, `@rdf_one_to_many`) that keeps the mapping co-located with the class definition, reducing boilerplate and improving readability.
- Automatic serialization and deserialization between Python objects and RDF graphs, including circular reference detection.
- A dynamic query repository that generates SPARQL queries at runtime from method names, supporting exact match, regex match, compound filters, pagination, count, and aggregation.
- Automatic SHACL shape generation from class metadata and validation via `pyshacl`.

pyrdm grew directly from research at the LambdaGeo Group at UFMA. Its predecessor, simpot (LambdaGeo Group, 2022), introduced a basic decorator approach to RDF mapping in Python. The practical experience of using simpot in the dbacademic-etl project (Costa et al., 2020) — a pipeline connecting open academic data from Brazilian institutions into RDF knowledge graphs — revealed the need for a richer mapping framework supporting relationships, deserialization, and validation. This motivated the development of RDFMapper as a Bachelor's thesis (Goiabeira, 2025), of which pyrdm is the packaged and maintained evolution.

Design and API

The central class is PyRDM, which acts both as a decorator factory and as the serialization engine. A typical mapping looks as follows:

```python
from rdflib import Namespace
from pyrdm import PyRDM, RDFRepository

EX = Namespace("http://example.org/")
FOAF = Namespace("http://xmlns.com/foaf/0.1/")

mapper = PyRDM()

@mapper.rdf_entity(EX.Person)
class Person:
    def __init__(self, uri, name: str, age: int = None):
        self.uri = uri
        self._name = name
        self._age = age

    @mapper.rdf_property(FOAF.name, minCount=1)
    def name(self): pass

    @mapper.rdf_property(FOAF.age)
    def age(self): pass
```

Serialization and deserialization are performed via mapper.to_rdf(obj) and mapper.from_rdf(graph, cls, uri). The RDFRepository class provides a dynamic query API:

```python
repo = RDFRepository(mapper, graph, Person)
repo.find_by_name(name="Felipe")
repo.find_by_name_like(name="Fel")
repo.count_by_name(name="Felipe")
repo.group_by_count(Person, "name", order="DESC")
```

SHACL shapes are generated automatically from the cardinality constraints declared in @rdf_property and validated via pyshacl (Sommer, 2024):

```python
conforms, _, report = mapper.validate(graph, entity_class=Person)
```


Acknowledgements

This work was developed as part of a Bachelor's thesis at the Federal University of Maranhão (UFMA), Brazil, within the LambdaGeo Research Group. The authors thank the UFMA Department of Computer Science for institutional support.

DRAFT

References

Bayer, M. (2012). *SQLAlchemy* (A. Brown & G. Wilson, Eds.). aosabook.org. http://aosabook.org/en/sqlalchemy.html

Bischof, S., Decker, B., Kurz, T., & Schandl, B. (2011). Empire: A JPA-based RDF object-mapper. *The Semantic Web: Research and Applications*, 336–350. https://doi.org/10.1007/978-3-642-21034-1_23

Costa, S. S., Sousa, M. V. D., Silva, M. L. da, Oliveira, E. C. de, & Guimaraes, J. V. M. (2020). DBacademic: Conectando os dados abertos das instituicoes de ensino do brasil. *Ciencia Da Informacao*, *49*(3).

Dodds, L. (2012). *RDFAlchemy: An ORM for RDF in python*. https://github.com/rdfalchemy/rdfalchemy

Eclipse Foundation. (2020). *Jakarta persistence 3.0* [Specification]. Eclipse Foundation. https://jakarta.ee/specifications/persistence/3.0/

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley. ISBN: 978-0321127426

Goiabeira, F. dos S. (2025). *Mapeamento declarativo para a web semantica: Desenvolvimento e avaliacao do framework RDFMapper para integracao objeto-RDF em python*.

Heath, T., & Bizer, C. (2011). *Linked data: Evolving the web into a global data space*. Morgan; Claypool Publishers. https://doi.org/10.2200/S00334ED1V01Y201102WBE001

LambdaGeo Group. (2022). *Simpot: Simple object-triple mapping for python*. https://github.com/LambdaGeo/simpot

Sommer, A. (2024). *pySHACL: A python validator for SHACL*. https://github.com/RDFLib/pySHACL

Team, R. (2023). *RDFLib: A python library for working with RDF* (Version 6.0). https://github.com/RDFLib/rdflib

W3C RDF Data Shapes Working Group. (2017). *Shapes constraint language (SHACL)*. https://www.w3.org/TR/shacl/

W3C RDF Working Group. (2014). *RDF 1.1 concepts and abstract syntax*. https://www.w3.org/TR/rdf11-concepts/

W3C SPARQL Working Group. (2013). *SPARQL 1.1 query language*. https://www.w3.org/TR/sparql11-query/

DRAFT